NAQLA

المختبر 3

المختبر 3 — انقلها إلى المعالجة غير المتزامنة

هندسة الواجهات الخلفية بالوكلاء الذكيين

هندسة الواجهات الخلفية بالوكلاء الذكيين – بناء أنظمة خلفية قابلة للتوسع بالذكاء الاصطناعي ومن أجله

المختبر 3 — انقلها إلى المعالجة غير المتزامنة

نظرة عامة

الهدف

تعمل واجهة برمجة (API) «شحنة» جيداً مع عميل واحد يضغط ببطء. اليوم ستجعلها تصمد تحت الحِمل: لا استعلامات N+1 (N+1 Queries)، ونقل العمل البطيء إلى طوابير المهام (Queues)، ولا حجز مزدوج عندما يعيد هاتف إرسال طلب، وتخزين مؤقت (Caching) للبيانات المرجعية، وحدود للمعدّل (Rate Limits). ستقوم **بالقياس قبل التغيير وبعده** باستخدام k6، ولن تصدّق أي ادعاء حول الأداء (منك أو من الوكيل) ما لم يدعمه رقم.

عادة اليوم: **الوكيل يقترح، والأرقام تحكم.** عندما يقول الوكيل «سيكون هذا أسرع»، يكون ردّك «أرني».

ما الذي ستبنيه

الجزء	مكانه
سكربت حِمل k6 وجدول قبل/بعد	`load/booking.js` ، `docs/plans/day3-perf.md`
إصلاح N+1 في `GET /shipments` مع اختبار لعدد الاستعلامات	`ShipmentController` ، `tests/Feature/PerformanceTest.php`
Horizon: `supervisor-1` للطوابير `dispatch` و `notifications` و `default` ؛ و `supervisor-ai` للطابور `ai`	`config/horizon.php`
الحدث `ShipmentBooked` والمستمع (Listener) الخاص به `QueueBookingJobs`	`app/Events/` ، `app/Listeners/`
`AssignVehicleJob` (أقفال الصفوف) و `SendBookingConfirmationJob`	`app/Jobs/`
`POST /api/dispatch/shipments/{reference}/assign`	تجاوز يدوي للموظفين فقط. `BookingService::assign()`
البرمجية الوسيطة `EnsureIdempotency` (الاسم المستعار `idempotent`) والنموذج `IdempotencyKey`	`app/Http/Middleware/` ، `app/Models/`

الجزء	مكانه
تخزين المدن والمسافات مؤقتاً	`CityController` ، `PricingService`
محدِّد المعدّل `api` (60 في الدقيقة) والبرمجية الوسيطة `AssignRequestId`	`app/Providers/AppServiceProvider.php` ، `bootstrap/app.php`

نقطة البداية

```
git checkout day2-end
git switch -c my-day3
sail up -d
sail artisan migrate:fresh --seed
sail artisan test
```

```
  Tests:    56 passed
```

ثبّت k6 على حاسوبك (`brew install k6` على macOS، و `winget install k6` على Windows)، أو استخدم Docker كما هو موضّح في الخطوة 1.

الميزانية الزمنية

الجزء	الخطوة	الدقائق
الجزء الأول (12:30-11:00)	1. تحميل البيانات وخط أساس k6	20
	2. إيجاد مشكلة N+1 يدوياً	15
	3. إصلاح N+1 بالاختبار أولاً	15
	4. تثبيت Horizon	10
	5. `ShipmentBooked` والمهام والأقفال	25
	نقطة التحقق 1	5
الجزء الثاني (15:45-14:15)	6. الإسناد اليدوي من المُرسِل	15
	7. مفاتيح عدم التكرار	25
	8. تخزين المدن والمسافات مؤقتاً	10
	9. حدود المعدّل ومعرّفات الطلبات	15
	10. قياس k6 بعد التغيير يدوياً	15
	نقطة التحقق 2 والإيداع	10

تحذير

يشغّل Sail التطبيق باستخدام `php artisan serve`، وهو ليس خادم ويب للإنتاج. أرقام k6 التي تحصل عليها لا معنى لها إلا **عند مقارنتها ببعضها على الحاسوب نفسه**. لا تقتبسها أبداً على أنها «شحنة تتحمّل N طلباً في الثانية».

الجزء الأول — قِس، وأصلح، وانتقل إلى اللاتزامن (11:00–12:30)

الخطوة 1 — تحميل البيانات وخط أساس k6 (20 دقيقة)

ماذا ولماذا. لا يمكنك تحسين ما لم تقِسه. قبل تغيير أي شيء، تحتاج إلى سكربت حِمل (Load Script) قابل للتكرار يمرّ على مسار الحجز: عرض السعر، ثم الحجز، ثم عرض القائمة.

🤖 **اسأل وكيلك الذكي**

الطلب للوكيل

خطّط أولاً، ثم انتظر موافقتي. فروق صغيرة.

أنشئ load/booking.js لأداة k6:

- يُؤخذ BASE_URL من __ENV.BASE_URL، والقيمة الافتراضية http://localhost؛ ويُؤخذ عدد المستخدمين الافتراضيين (VUs) من __ENV.VUS، والقيمة الافتراضية 20
- setup(): نفّذ GET /api/cities وأعِد معرّفات المدن
- الدالة الافتراضية: في تكرارها الأول فقط، يسجّل كل مستخدم افتراضي حسابه الخاص عبر POST /api/auth/register (ببريد إلكتروني فريد باستخدام uuidv4 من https://jslib.k6.io/k6-utils/1.4.0/index.js) ويحتفظ بالرمز المميز (Token). في كل تكرار: POST /api/quotes (مدينتان عشوائيتان، vehicle_type عشوائي، weight_kg بين 100 و1100)، ثم POST /api/shipments مع quote_id ذاك pickup_at في الغد، ثم GET /api/shipments. ضع الوسوم {'name: 'quote'} و{name:'book''} و{'name: 'list'} على الطلبات. افحص رموز الحالة بـ check() (201، 201، 200). ونفّذ sleep(3) في النهاية.
- أرسل الترويسة Idempotency-Key (uuidv4) مع POST /api/shipments (غير مستخدمة اليوم حتى الخطوة 7، ولا ضرر منها الآن).
- options: مراحل تصاعد خلال 20 ثانية، وثبات لمدة دقيقة، وتنازل خلال 10 ثوانٍ؛ والعتبات (Thresholds): http_req_failed بمعدّل أقل من 0.01، و p(95) أقل من 300 لـ {name:list}، و p(95) أقل من 500 لـ {name:book}.

لا تغيّر أي شيفرة في التطبيق.

بالإنجليزية:

```
Plan first, then wait for my approval. Small diffs.

Create load/booking.js for k6:
- BASE_URL from __ENV.BASE_URL, default http://localhost;
  number of VUs from __ENV.VUS, default 20
- setup(): GET /api/cities and return the city ids
- default function: on its FIRST iteration each VU registers its own
  account via POST /api/auth/register (unique email using uuidv4 from
  https://jslib.k6.io/k6-utils/1.4.0/index.js) and keeps the token.
  Every iteration: POST /api/quotes (two random cities, random
  vehicle_type, weight_kg 100-1100), POST /api/shipments with that
  quote_id and pickup_at tomorrow, then GET /api/shipments.
  Tag the requests {name: 'quote'}, {name: 'book'}, {name: 'list'}.
  check() status codes (201, 201, 200). sleep(3) at the end.
- Send an Idempotency-Key header (uuidv4) on POST /api/shipments
  (unused today until Step 7, harmless now).
- options: stages ramp up 20s, hold 1m, ramp down 10s; thresholds
  http_req_failed rate<0.01, p(95)<300 for {name:list},
  p(95)<500 for {name:book}.
Do not change any application code.
```

ما الذي تتحقق منه.

- جيد: `sleep(3)` في كل تكرار. هكذا يرسل كل مستخدم 3 طلبات كل 3 ثوانٍ أو أكثر، فيبقى تحت حدّ المعدّل البالغ 60 طلباً في الدقيقة الذي ستضيفه في الخطوة 9. من دون ذلك، سيمتلئ تشغيل «بعد» بأخطاء 429.
- جيد: العتبات محددة لكل وسم، مثلاً `'http_req_duration{name:list}': ['p(95)<300']`. لا يطبع k6 أرقاماً لكل وسم إلا للمقاييس التي لها عتبات.
- علامة تحذير: السكربت يسجّل مستخدماً جديداً في كل تكرار. هذا يقيس bcrypt لا واجهتك البرمجية. حساب واحد لكل مستخدم افتراضي يكفي.
- علامة تحذير: الوكيل يخترع واجهات k6 غير موجودة. تأتي `http` و`check` و`sleep` و`group` من `k6/http` و`k6`؛ ولا يوجد شيء اسمه `k6.request()`.

شغّل قياس خط الأساس (Baseline) (قاعدة البيانات مُعبّأة بالبيانات منذ نقطة البداية):

```
k6 run load/booking.js
```

إن لم يكن k6 مثبتاً محلياً، فاستخدم Docker (يصل `host.docker.internal` إلى Sail من داخل الحاوية):

```
docker run --rm -i -e BASE_URL=http://host.docker.internal grafana/k6 run - < load/booking.js
```

تبدو نهاية التقرير هكذا (ستختلف أرقامك):

```
  × http_req_duration{name:book}...: avg=160ms  p(95)=520ms
  × http_req_duration{name:list}...: avg=240ms  p(95)=410ms
    http_req_failed................: 0.00%
    http_reqs......................: 1240   12.4/s
```

أنشئ `docs/plans/day3-perf.md` **يدوياً** وانسخ إليه قيمتَي p95، و`http_req_failed`، وإجمالي الطلبات في الثانية، تحت عنوان "Baseline". ثم أودِع (Commit):

```
git add load docs/plans/day3-perf.md
git commit -m "chore: k6 baseline"
```

الخطوة 2 – إيجاد مشكلة N+1 يدوياً (15 دقيقة)

ماذا ولماذا. النقطة `GET /api/shipments` هي أبطأ نقطة نهاية (Endpoint) لديك. ستكتشف السبب **من دون الوكيل**، لأن «عدّ الاستعلامات» مهارة تحتاجها حين لا يكون الوكيل موجوداً.

> **تحذير**
>
> **يدوياً.** لا تطلب من الوكيل إيجاد المشكلة. جِدها أنت، ثم دع الوكيل يصلحها.

افتح Tinker وعُدّ الاستعلامات لعمل طلب واحد:

```
sail artisan tinker
```

```
> DB::enableQueryLog();
> $u = App\Models\User::where('email', 'customer@shahna.test')->first();
> App\Http\Resources\ShipmentResource::collection($u->shipments()->latest()->paginate(15))
      ->resolve();
> count(DB::getQueryLog());
= 32
> collect(DB::getQueryLog())->pluck('query')->take(6);
```

سترى الاستعلام نفسه `select * from "cities" where "cities"."id" = ? limit 1` يتكرر مرة بعد مرة: استعلام لكل شحنة من أجل `origin` وآخر من أجل `destination`. ومع استعلام عدّ الترقيم (Pagination) واستعلام الصفحة نفسها يصبح المجموع **32 = 15 × 2 + 2** استعلاماً لصفحة واحدة. شحنات العرض المُعبّأة ليس لها مركبة بعد؛ وحين تُسنَد المركبات، يضيف `vehicle` استعلاماً ثالثاً لكل شحنة. هذه هي مشكلة N+1.

طريقة ثانية لرؤيتها هي جعل Laravel يرفض التحميل الكسول (Lazy Loading) أثناء التطوير. أضف في `AppServiceProvider::boot()`:

```php
use Illuminate\Database\Eloquent\Model;

Model::preventLazyLoading(! app()->isProduction());
```

الآن استدعِ نقطة النهاية:

```bash
curl -s http://localhost/api/shipments \
  -H "Accept: application/json" -H "Authorization: Bearer $TOKEN" | jq .message
```

```
"Attempted to lazy load [origin] on model [App\\Models\\Shipment] but lazy loading is
        disabled."
```

احذف سطر `preventLazyLoading` مجدداً بعد أن ترى الخطأ. إنه أداة تشخيص جيدة، لكن مسارات أخرى في الشيفرة تحمّل العلاقات كسولاً عن قصد (مثلاً يقرأ `SendBookingConfirmationJob` القيمة `$shipment` ثم `->user`). بدلاً من ذلك، يحمي الحلّ المرجعي `GET /api/shipments` باختبار لعدد الاستعلامات (الخطوة 3)، فتؤدي أي مشكلة N+1 جديدة هناك إلى فشل اختبار بدلاً من إبطاء بيئة الإنتاج.

الخطوة 3 – إصلاح N+1 بالاختبار أولاً (15 دقيقة)

🤖 **اسأل وكيلك الذكي**

الطلب للوكيل

خطّط أولاً، ثم انتظر موافقتي. الاختبار أولاً.

1. في tests/Feature/PerformanceTest.php أضف اختباراً يعدّ استعلامات GET /api/shipments (DB::enableQueryLog() / count(DB::getQueryLog())) لمستخدم لديه شحنتان، ثم يعدّها مرة أخرى بعد إضافة 10 شحنات أخرى، ويؤكد أن العددين متساويان. شغّله؛ يجب أن يفشل.
2. أصلح ShipmentController@index بالتحميل المسبق (Eager Loading): with(['origin', 'destination', 'vehicle']). لا تغيّر شكل مخرجات ShipmentResource.
3. القائمة تُرشَّح حسب user_id وتُرتَّب حسب created_at. تحقّق من أن ترحيل الشحنات في اليوم الأول يحتوي بالفعل على فهرس على (user_id, created_at)؛ ولا تُضِف فهرساً ثانياً.

شغّل مجموعة الاختبارات كاملة وأبلغني بعدد الاستعلامات قبل الإصلاح وبعده.

بالإنجليزية:

```
Plan first, then wait for my approval. Test first.

1. In tests/Feature/PerformanceTest.php add a test that counts the
   queries of GET /api/shipments (DB::enableQueryLog() /
   count(DB::getQueryLog())) for a user with 2 shipments, then again
   after adding 10 more, and asserts both counts are equal. Run it; it
   must fail.
2. Fix ShipmentController@index with eager loading: with(['origin',
   'destination', 'vehicle']). Do not change ShipmentResource's output
   shape.
3. The list filters by user_id and orders by created_at. Check that the
   Day 1 shipments migration already has an index on (user_id,
   created_at); do not add a second one.
Run the full test suite and report the query count before and after.
```

ما الذي تتحقق منه.

- جيد: يستخدم `ShipmentResource` الدالة `$this->whenLoaded('vehicle')` أو أن المتحكم يحمّل العلاقات مسبقاً دائماً، فلا يطلق المورد أي استعلام من تلقاء نفسه.
- جيد: يقارن الاختبار عددين (شحنتان مقابل 12) بدلاً من رقم سحري، فيفشل بمجرد أن يكبر عدد الاستعلامات مع حجم الصفحة.
- علامة تحذير: الإصلاح يخزّن القائمة كاملة في Redis. هذا يخفي مشكلة N+1 بدلاً من إصلاحها، ويقدّم بيانات قديمة بعد الإلغاء.
- علامة تحذير: خُفّف شرط عدد الاستعلامات حتى ينجح الاختبار (مثلاً `toBeLessThan(50)`).

افحص خطة تنفيذ الاستعلام الحقيقي في PostgreSQL يدوياً (العميل المُعبّأ هو المستخدم 1):

```
sail psql -c "EXPLAIN SELECT * FROM shipments WHERE user_id = 1 ORDER BY created_at DESC LIMIT
        15;"
```

```
 Limit  (cost=0.28..12.45 rows=15 width=...)
   →  Index Scan Backward using shipments_user_id_created_at_index on shipments ...
```

`Index Scan` هو ما تريده. أما `Seq Scan` على جدول كبير فيعني أن الفهرس لا يُستخدم.

الخطوة 4 — تثبيت Horizon (10 دقائق)

ماذا ولماذا. يشغّل Horizon عمّال طوابير Redis (Queue Workers) ويراقبهم. ستستخدم أربعة طوابير، وتضع `ai` على مشرف (Supervisor) خاص به، حتى لا يتمكن عمل الذكاء الاصطناعي البطيء (اليوم الرابع) من تعطيل إسناد المركبات أبداً.

الحزمة `laravel/horizon` موجودة بالفعل في `composer.json` منذ `day1-start`. انشر ملف إعداداتها ومزوّد الخدمة (Service Provider) الخاص بها:

```
sail artisan horizon:install
```

تحقّق في `.env` من `QUEUE_CONNECTION=redis`. ثم عدّل `defaults` في `config/horizon.php` **يدوياً**: يعالج `supervisor-1` ثلاثة طوابير بترتيب الأولوية، ويتولى مشرف ثانٍ الطابور `ai` وحده:

```
'defaults' ⇒ [
    'supervisor-1' ⇒ [
        'connection' ⇒ 'redis',
        'queue' ⇒ ['dispatch', 'notifications', 'default'],
        'balance' ⇒ 'auto',
        // ...keep the other generated keys...
        'tries' ⇒ 3,
        'timeout' ⇒ 60,
    ],
    // LLM calls are slow and expensive: give them their own workers and a longer timeout.
    'supervisor-ai' ⇒ [
        'connection' ⇒ 'redis',
        'queue' ⇒ ['ai'],
        'balance' ⇒ 'simple',
        'maxProcesses' ⇒ 1,
        'memory' ⇒ 128,
        'tries' ⇒ 2,
        'timeout' ⇒ 180,
    ],
],
```

اترك `environments` كما وُلّدت: فهي تضبط فقط `maxProcesses` (10 في `production`، و3 في `local`) للمشرف `supervisor-1`.

شغّل Horizon في **طرفية ثانية** واتركه يعمل:

```
sail artisan horizon
```

```
  INFO  Horizon started successfully.
```

افتح `http://localhost/horizon`. لوحة التحكم مفتوحة في بيئة `local` فقط؛ أما في الإنتاج فيتحكم في الوصول إليها الحارس (Gate) `viewHorizon` في `app/Providers/HorizonServiceProvider.php` (الحلّ المرجعي يسمح للمسؤولين فقط). كما يجدول الحلّ المرجعي `horizon:snapshot` كل خمس دقائق في `routes/console.php` حتى تمتلئ الرسوم البيانية للمقاييس.

> **نصيحة**
>
> لا يعيد Horizon تحميل الشيفرة. بعد تغيير صنف مهمة، أوقف Horizon بالضغط على `Ctrl+C` وشغّله من جديد، وإلا فستصحّح أخطاء شيفرة الأمس.

الخطوة 5 — `ShipmentBooked` والمهام والأقفال (25 دقيقة)

ماذا ولماذا. الحجز حالياً ينفّذ كل شيء داخل طلب HTTP. إسناد مركبة وإرسال تأكيد عمليتان بطيئتان وقد تفشلان، لذا تنتقلان إلى الطوابير. الطلب يحجز الشحنة فقط ويطلق `ShipmentBooked`.

ثلاث قواعد مهمة هنا:

1. **أرسِل بعد الإيداع.** يطلق `BookingService::book()` الحدث `ShipmentBooked` *داخل* `DB::transaction` الخاصة به. إن التقط عامل المهمة قبل إيداع تلك المعاملة (Transaction)، فلن يرى الشحنة بعد (أو سيرى شحنة محفوظة جزئياً). جعل الحدث يطبّق `ShouldDispatchAfterCommit` يؤخّره حتى الإيداع، ويُسقطه إن تراجعت المعاملة.
2. **أعِد القراءة تحت قفل.** تحمل المهمة الموضوعة في الطابور نسخة متسلسلة (Serialized) من الشحنة. وحين يحين تشغيلها، قد يكون العميل ألغاها أو يكون المُرسِل (Dispatcher) قد أسندها. لذلك تفتح المهمة معاملة وتعيد قراءة الصف باستخدام `Shipment::lockForUpdate()->find(...)` قبل أن تقرر أي شيء. ويفعل `BookingService::cancel()` الشيء نفسه منذ اليوم الثاني.
3. **اقفل صف المركبة.** عاملان يُسندان شحنتين في اللحظة نفسها قد يختاران الشاحنة الصغيرة الشاغرة نفسها. استدعاء `lockForUpdate()` على استعلام المركبة، داخل المعاملة، يجعل العامل الثاني ينتظر حتى يودِع الأول.

الحجز ينقل الشحنة بالفعل `pending → confirmed` (حدثان من نوع `ShipmentEvent`، وتعيد الواجهة `"status": "confirmed"`). تسمح آلة الحالات (State Machine) بالانتقال `confirmed → assigned`، لذا تقتصر المهمة على الإسناد، وفقط إذا كانت الشحنة المُعاد قراءتها لا تزال `confirmed`. وإن لم تتوفر مركبة شاغرة، تبقى الشحنة `confirmed` بانتظار المُرسِل.

🤖 **اسأل وكيلك الذكي**

الطلب للوكيل

خطّط أولاً، ثم انتظر موافقتي. الاختبارات أولاً. فروق صغيرة.

1. App(يحمل الشحنة Shipment)، ويطبّق ShouldDispatchAfterCommit. أطلقه من BookingService::book() بعد تأكيد الشحنة، داخل المعاملة الموجودة.
2. مستمع AppيرسلAssignVehicleJob إلى الطابور "dispatch"
   - SendBookingConfirmationJob إلى الطابور "notifications"
3. AssignVehicleJob(Shipment):
   - داخل DB::transaction: أعِد قراءة الشحنة باستخدام Shipment::lockForUpdate()->find(id)؛ وإن لم تكن حالتها confirmed (ملغاة، أو مُسندة مسبقاً)، فاخرج. يجب أن تكون المهمة آمنة إذا شُغّلت مرتين.
   - ابحث عن أصغر مركبة Vehicle متاحة يكون فيها type مساوياً لـ vehicle_type الخاص بالشحنة، capacity_kg >= weight_kg، current_city_id مساوياً لمدينة المنشأ، مع orderBy('capacity_kg')، وباستخدام lockForUpdate()
   - إن وُجدت: حالة المركبة busy، ويُضبط vehicle_id للشحنة، وانتقال confirmed -> assigned باستخدام transitionTo() (مع ShipmentEvent "Vehicle <plate> assigned.")
   - إن لم توجد: Log::warning مع المرجع، واترك الحالة confirmed
   - أضف ترحيلاً (Migration) بفهرس على vehicles (current_city_id, type, status) يطابق شرط WHERE ذاك
4. SendBookingConfirmationJob: تأكيد نصي عادي عبر Mail::raw إلى المستخدم (المرجع، ووقت الاستلام، والسعر)؛ MAIL_MAILER=log أثناء التطوير.

الاختبارات:

- الحجز يرسل ShipmentBooked (Event::fake)
- الحجز يدفع المهمتين إلى الطابورين الصحيحين (Queue::fake، Queue::assertPushedOn)
- تختار AssignVehicleJob أصغر مركبة مناسبة في مدينة المنشأ، متجاوزة المركبات المشغولة، وذات النوع الخاطئ، والصغيرة جداً، والموجودة في مدينة أخرى
- لا مركبة -> الحالة confirmed، vehicle_id قيمته null
- الشحنة أُلغيت قبل تشغيل المهمة -> المهمة لا تفعل شيئاً

اختبارات الحجز الحالية تبقى دون تغيير: حالة الاستجابة لا تزال "confirmed".

بالإنجليزية:

```
Plan first, then wait for my approval. Tests first. Small diffs.

1. App\Events\ShipmentBooked (carries the Shipment), implementing
   ShouldDispatchAfterCommit. Fire it from BookingService::book(), after
   the shipment is confirmed, inside the existing transaction.
2. Listener App\Listeners\QueueBookingJobs that dispatches:
   - AssignVehicleJob on the "dispatch" queue
   - SendBookingConfirmationJob on the "notifications" queue
3. AssignVehicleJob(Shipment):
   - in DB::transaction: re-read the shipment with
     Shipment::lockForUpdate()->find(id); if its status is not
     confirmed (cancelled, or already assigned), return. The job must be
     safe to run twice.
   - find the smallest available Vehicle with type = shipment
     vehicle_type, capacity_kg ⩾ weight_kg, current_city_id = origin,
     orderBy('capacity_kg'), using lockForUpdate()
   - if found: vehicle status busy, shipment vehicle_id set, transition
     confirmed → assigned with transitionTo() (ShipmentEvent
     "Vehicle <plate> assigned.")
   - if none: Log::warning with the reference and leave status confirmed
   - add a migration with an index on vehicles (current_city_id, type,
     status) to match that WHERE clause
4. SendBookingConfirmationJob: Mail::raw plain-text confirmation to the
   user (reference, pickup time, price); MAIL_MAILER=log in development.
Tests:
- booking dispatches ShipmentBooked (Event::fake)
- booking pushes both jobs on the right queues (Queue::fake,
  Queue::assertPushedOn)
- AssignVehicleJob picks the smallest vehicle that fits in the origin
  city, skipping busy, wrong type, too small and wrong-city vehicles
- no vehicle → status confirmed, vehicle_id null
- shipment cancelled before the job runs → job does nothing
Existing booking tests stay unchanged: the response status is still
"confirmed".
```

ما الذي تتحقق منه.

- جيد: يُستدعى `lockForUpdate()` **داخل** `DB::transaction`. خارج المعاملة يُحرَّر القفل فوراً ولا يفعل شيئاً.
- جيد: تعيد المهمة قراءة الشحنة باستخدام `lockForUpdate()` بدلاً من الوثوق بـ `$this->shipment` المهام في الطابور تسلسل معرّف النموذج وتعيد جلبه عند بدء المهمة، لكن من دون قفل قد تتغير الحالة بين تلك القراءة والتحديث (مثلاً وصول طلب إلغاء في اللحظة نفسها).`->status`
- جيد: `ShipmentBooked implements ShouldDispatchAfterCommit`. من دونه تظل الاختبارات ناجحة (لأنها تعمل داخل معاملة واحدة مع طابور مزيّف)، لذا اقرأ صنف الحدث بنفسك.
- علامة تحذير: `Vehicle::where(...)->get()->first()` ثم `lockForUpdate()` ← يجب أن يكون القفل جزءاً من الاستعلام الذي يختار الصف.

- علامة تحذير: المهمة تستدعي `sleep()` أو تعيد المحاولة إلى ما لا نهاية. استخدم `tries` في Horizon و`$backoff` في المهمة.
- علامة تحذير: الانتقال `pending → assigned` في خطوة واحدة، أو تغييرات حالة تتجاوز `canTransitionTo`.

أثبت أن القفل يعمل **يدوياً**. في Tinker، أنشئ سيناريو فيه شحنتان ومركبة مطابقة واحدة فقط، ثم أرسل المهمتين بينما Horizon يعمل (3 عمليات):

```
> $v = App\Models\Vehicle::where('status','available')->first();
> App\Models\Vehicle::where('id','<>',$v->id)->where('type',$v->type)
      ->where('current_city_id',$v->current_city_id)->update(['status'=>'maintenance']);
> $a = App\Models\Shipment::factory()->create(['vehicle_type'=>$v->type,'origin_city_id'=>$v
      ->current_city_id,'weight_kg'=>100]);
> $b = App\Models\Shipment::factory()->create(['vehicle_type'=>$v->type,'origin_city_id'=>$v
      ->current_city_id,'weight_kg'=>100]);
> App\Jobs\AssignVehicleJob::dispatch($a)->onQueue('dispatch');
      App\Jobs\AssignVehicleJob::dispatch($b)->onQueue('dispatch');
> [$a->fresh()->status, $b->fresh()->status];
```

```
= [App\Enums\ShipmentStatus::Assigned, App\Enums\ShipmentStatus::Confirmed]
```

شحنة واحدة بالضبط تحصل على المركبة. افحص لوحة Horizon: كلتا المهمتين تحت **Completed**، ولا شيء تحت **Failed**.

نقطة تحقق

نقطة التحقق 1 (12:30)

```
sail artisan test
```

```
  Tests:    60+ passed
```

صفر إخفاقات، وعدد اختبارات أكبر من الـ 56 التي بدأت بها. ثم احجز شحنة عبر curl (Horizong يعمل) وافحص تتبّعها بعد بضع ثوانٍ:

```
curl -s http://localhost/api/shipments/$REF/tracking \
  -H "Accept: application/json" -H "Authorization: Bearer $TOKEN" | jq
        '[.data[].status]'
```

```
["pending", "confirmed", "assigned"]
```

النتيجة `["pending", "confirmed"]` صحيحة أيضاً إن لم تكن هناك مركبة شاغرة من ذلك النوع في مدينة المنشأ.

```
git add -A
git commit -m "feat: async booking with vehicle assignment"
```

الجزء الثاني – عدم التكرار، والتخزين المؤقت، والحدود (14:15–15:45)

الخطوة 6 – الإسناد اليدوي من المُرسِل (15 دقيقة)

ماذا ولماذا. عندما لا تجد المهمة التلقائية مركبة، يُسند المُرسِل واحدة يدوياً. هذه هي مشكلة الأقفال نفسها التي في المهمة، لذا ينبغي أن يعيش منطق الإسناد في مكان **واحد** يستدعيه كلاهما.

🤖 **اسأل وكيلك الذكي**

الطلب للوكيل

خطّط أولاً، ثم انتظر موافقتي. الاختبارات أولاً.

POST /api/dispatch/shipments/{shipment:reference}/assign، وجسم الطلب {vehicle_id}.

- ShipmentPolicy::assign: للموظفين فقط (dispatcher أو admin، عبر User::isStaff()).
- FormRequest باسم AssignVehicleRequest: vehicle_id مطلوب، وعدد صحيح، exists:vehicles,id.
- انقل منطق «أسنِد هذه المركبة إلى هذه الشحنة» من AssignVehicleJob إلى BookingService::assign(Shipment, Vehicle, note)، ليستخدمه كلٌّ من المهمة ونقطة النهاية هذه. داخل DB::transaction خاصة به، أعِد قراءة الصفّين كليهما باستخدام lockForUpdate() وأعِد فحصهما.
- 422 مع errors.vehicle_id إذا كانت المركبة غير متاحة، أو من نوع خاطئ، أو صغيرة جداً؛ و422 مع errors.status إذا لم تكن الشحنة confirmed أو assigned. إعادة إسناد شحنة مُسندة تحرّر المركبة القديمة.
- أعِد 200 مع ShipmentResource (متضمّناً المركبة).

الاختبارات: نجاح المُرسِل؛ إعادة الإسناد تحرّر المركبة القديمة؛ العميل -> 403؛ مركبة مشغولة أو غير مناسبة -> 422؛ شحنة ملغاة -> 422. يجب أن تظل اختبارات المهمة الحالية ناجحة دون تغيير.

بالإنجليزية:

```
Plan first, then wait for my approval. Tests first.

POST /api/dispatch/shipments/{shipment:reference}/assign, body {vehicle_id}.
- ShipmentPolicy::assign: staff only (dispatcher or admin, User::isStaff()).
- FormRequest AssignVehicleRequest: vehicle_id required, integer,
  exists:vehicles,id.
- Move the "assign this vehicle to this shipment" logic out of
  AssignVehicleJob into BookingService::assign(Shipment, Vehicle, note),
  used by both the job and this endpoint. In its own DB::transaction,
  re-read BOTH rows with lockForUpdate() and re-check them.
- 422 with errors.vehicle_id if the vehicle is not available, wrong type,
  or too small; 422 with errors.status if the shipment is not confirmed
  or assigned. Re-assigning an assigned shipment frees the old vehicle.
- Return 200 with ShipmentResource (vehicle included).
Tests: dispatcher success; re-assign frees the old vehicle; customer →
403; busy or unsuitable vehicle → 422; cancelled shipment → 422.
Existing job tests must still pass unchanged.
```

ما الذي تتحقق منه.

- جيد: يُظهر `git diff tests/` اختبارات جديدة فقط؛ واختبارات المهمة من الخطوة 5 لم تُمسّ وما زالت خضراء.
- علامة تحذير: لكلٍّ من نقطة النهاية والمهمة نسخته الخاصة من فحوصات المركبة.
- علامة تحذير: المسار ليس خلف `auth:sanctum`، أو لا يوجد `Gate::authorize('assign', $shipment)` إطلاقاً.
- ملاحظة: يتحقق FormRequest من `vehicle_id` قبل أن يُنفَّذ فحص السياسة (Policy) في المتحكم، لذا فالعميل الذي يرسل معرّفاً غير موجود يحصل على 422 بدلاً من 403. الحلّ المرجعي يقبل ذلك؛ وإضافة دالة `authorize()` إلى `AssignVehicleRequest` تسدّ هذه الثغرة.

الخطوة 7 — مفاتيح عدم التكرار (25 دقيقة)

ماذا ولماذا. هاتف على اتصال 3G ضعيف في سبها يرسل `POST /api/shipments`، فتضيع الاستجابة، ويعيد التطبيق المحاولة. من دون حماية، قد يُحاسَب العميل على شاحنتين. قيد التفرّد على `quote_id` ينقذك في نقطة النهاية هذه، لكن العميل يتلقى خطأ 422 مربكاً. النمط المعتمد في الصناعة هو ترويسة **Idempotency-Key** (مفتاح عدم التكرار): يرسل العميل مفتاحاً فريداً لكل إجراء منطقي؛ ويخزّن الخادم الاستجابة الأولى ويعيد تقديمها عند إعادة المحاولة.

القواعد في «شحنة»:

الحالة	الاستجابة
لا توجد ترويسة `Idempotency-Key`	400 `{"message": "The Idempotency-Key header is required."}`
مفتاح جديد	نفّذ الطلب، وخزّن الحالة + الجسم، وأعِدهما
المفتاح نفسه، والمستخدم نفسه، والجسم نفسه	أعِد تقديم الحالة والجسم المخزّنين، مع الترويسة `Idempotent-Replayed: true`
المفتاح نفسه، والمستخدم نفسه، وجسم مختلف	422 `{"message": "This Idempotency-Key was already used for a different request."}`
المفتاح نفسه، والطلب الأول لا يزال قيد التنفيذ	409 `{"message": "A request with this Idempotency-Key is still being processed."}`
المفتاح نفسه، والطلب الأول بدأ قبل أكثر من 60 ثانية ولم ينتهِ أبداً	اعتبره متروكاً (العامل تعطّل): نفّذ الطلب مجدداً
انتهى الطلب الأول بخطأ 5xx	يُحذف المفتاح، فيستطيع العميل إعادة المحاولة بالمفتاح نفسه

المفاتيح محصورة بكل مستخدم على حدة، وتنتهي صلاحيتها بعد يوم واحد.

قاعدة الستين ثانية مهمة. من دونها، فإن عملية PHP تموت بين «حجز المفتاح» و«تخزين الاستجابة» تترك صفاً بلا استجابة إلى الأبد، وكل إعادة محاولة لذلك الحجز تحصل على 409 حتى تنتهي صلاحية المفتاح بعد يوم.

🤖 **اسأل وكيلك الذكي**

الطلب للوكيل

خطّط أولاً، ثم انتظر موافقتي. الاختبارات أولاً، وفروق صغيرة.

1. ترحيل + نموذج IdempotencyKey: key (string)، user_idg (مفتاح أجنبي FK)، request_hashg (char 64)، response_statusg (small int يقبل null)، response_bodyg (text يقبل null)، expires_atg (مُفهرس). فهرس فريد على (user_id, key). اجعل النموذج MassPrunable للصفوف التي تجاوزت expires_at؛ ويُجدوَل model:prune يومياً في routes/console.php.
2. برمجية وسيطة App، باسم مستعار "idempotent" مسجّل في bootstrap/app.php، تُطبَّق فقط على POST /api/shipments.
   - ترويسة مفقودة، أو أطول من 255 حرفاً -> 400
   - request_hash = sha256 للطريقة + المسار + الجسم الخام
   - احجز المفتاح قبل تنفيذ الطلب باستخدام IdempotencyKey::createOrFirst(['user_id', 'key'], [...]) (يعتمد على الفهرس الفريد)، ثم خزّن الحالة + الجسم بعد ذلك
   - المفتاح نفسه + بصمة (Hash) مختلفة -> 422؛ المفتاح نفسه + لا استجابة بعد -> 409؛ المفتاح نفسه + استجابة مخزّنة -> أعِد تقديمها مع الترويسة Idempotent-Replayed: true
   - المفتاح المنتهية صلاحيته، أو الذي لا استجابة له وآخر تحديث له قبل أكثر من IN_FLIGHT_TIMEOUT_SECONDS = 60، يُحذف ويُحجز من جديد
   - عند استجابة 5xx احذف الصف حتى يتمكن العميل من إعادة المحاولة
3. حدّث load/booking.js فقط عند الحاجة (فهو يرسل الترويسة بالفعل).

الاختبارات: كل صف من الجدول، بما فيه صف الطلب المتروك (استخدم this->travel())؛ المفاتيح لكل مستخدم (المستخدم B يستطيع استخدام مفتاح المستخدم A)؛ إعادة التقديم تعيد الجسم نفسه ولا توجد شحنة Shipment ثانية.

بالإنجليزية:

```
Plan first, then wait for my approval. Tests first, small diffs.

1. Migration + model IdempotencyKey: key (string), user_id (FK),
   request_hash (char 64), response_status (nullable small int),
   response_body (nullable text), expires_at (indexed). Unique index on
   (user_id, key). Make the model MassPrunable for rows past expires_at;
   model:prune is scheduled daily in routes/console.php.
2. Middleware App\Http\Middleware\EnsureIdempotency, alias "idempotent"
   registered in bootstrap/app.php, applied ONLY to POST /api/shipments.
   - missing header, or longer than 255 chars → 400
   - request_hash = sha256 of method + path + raw body
   - claim the key BEFORE running the request with
     IdempotencyKey::createOrFirst(['user_id', 'key'], [...]) (it relies
     on the unique index), then store status + body afterwards
   - same key + different hash → 422; same key + no response yet → 409;
     same key + stored response → replay it with header
     Idempotent-Replayed: true
   - a key that is expired, or has no response and was last updated more
     than IN_FLIGHT_TIMEOUT_SECONDS = 60 ago, is deleted and claimed again
   - on a 5xx response delete the row so the client can retry
3. Update load/booking.js only if needed (it already sends the header).
Tests: each row of the table, including the abandoned-request row (use
$this→travel()); keys are per user (user B can use user A's key);
replay returns the same body and no second Shipment exists.
```

ما الذي تتحقق منه.

- جيد: يُدرَج الصف **قبل** تشغيل المتحكم. أسلوب «تحقّق من وجوده، ثم أدرِج لاحقاً» فيه حالة تسابق (Race Condition): تصل محاولتان معاً، فلا ترى أيٌّ منهما شيئاً، فتحجز كلتاهما.
- جيد: اختبار ينشئ مفتاحاً بلا استجابة، وينتقل زمنياً دقيقتين، ويتحقق من أن الطلب التالي يُنفَّذ (201) بدلاً من الحصول على 409.
- جيد: استجابة إعادة التقديم مطابقة للاستجابة الأولى بايتاً ببايت (`reference` نفسه).
- علامة تحذير: المفتاح عام بدلاً من أن يكون لكل مستخدم. يستطيع المستخدم B عندها إعادة تقديم استجابة المستخدم A ورؤية شحنته.
- علامة تحذير: البرمجية الوسيطة تُطبَّق على كل المسارات، بما فيها `GET`. طلبات GET عديمة التكرار أصلاً.
- علامة تحذير: الجسم المخزّن محفوظ في Redis من دون صف في قاعدة البيانات. إن مُسح Redis، تختفي الحماية من الحجز المزدوج بصمت.

اختبرها يدوياً بإعادة محاولة، ثم بطلبين متوازيين:

```
KEY=$(uuidgen)
Q=$(curl -s -X POST http://localhost/api/quotes -H "Accept: application/json" \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"origin_city_id":1,"destination_city_id":3,"weight_kg":500,"vehicle_type":"van"}' | jq
      .data.id)

for i in 1 2; do
  curl -s -i -X POST http://localhost/api/shipments \
    -H "Accept: application/json" -H "Authorization: Bearer $TOKEN" \
    -H "Content-Type: application/json" -H "Idempotency-Key: $KEY" \
    -d "{\"quote_id\":$Q,\"pickup_at\":\"2026-10-26T08:00:00Z\"}" | grep -E "HTTP/|Idempotent-
      Replayed|reference"
done
```

```
HTTP/1.1 201 Created
{"data":{"reference":"SHN-P4M9QX",...
HTTP/1.1 201 Created
Idempotent-Replayed: true
{"data":{"reference":"SHN-P4M9QX",...
```

أرسل المفتاح نفسه مرة أخرى مع قيمة `pickup_at` مختلفة: ستحصل على 422 والرسالة `"This Idempotency-Key was already used for a different request."`

المرجع نفسه مرتين، وشحنة واحدة في قاعدة البيانات. والآن من دون الترويسة:

```
{"message":"The Idempotency-Key header is required."}
```

الخطوة 8 — تخزين المدن والمسافات مؤقتاً (10 دقائق)

ماذا ولماذا. المدن والمسافات لا تتغير تقريباً أبداً، لكنها تُقرأ مع كل عرض سعر. إنها مرشّحة مثالية للتخزين المؤقت. والجزء الصعب في التخزين المؤقت ليس التخزين، بل **الإبطال (Invalidation)**.

🤖 **اسأل وكيلك الذكي**

الطلب للوكيل

خطّط أولاً، ثم انتظر موافقتي. الاختبارات أولاً.

1. GET /api/cities: Cache::remember(City::CACHE_KEY, now()->addDay(), …) مع City::CACHE_KEY = "cities".
2. PricingService::distanceKm(): خزّن نتيجة البحث عن الكيلومترات لكل زوج مؤقتاً بالمفتاح "distance:{from}:{to}" لمدة أسبوع (مسافات الطرق لا تتغير).
3. الإبطال: في City::booted()، احذف City::CACHE_KEY من الذاكرة المؤقتة عند حفظ مدينة City أو حذفها.

الاختبارات: طلب GET /api/cities الثاني لا ينفّذ أي استعلام (Cache تستخدم المخزن array في الاختبارات)؛ وتحديث name_en لمدينة يغيّر استجابة /api/cities التالية.

بالإنجليزية:

```
Plan first, then wait for my approval. Tests first.

1. GET /api/cities: Cache::remember(City::CACHE_KEY, now()→addDay(), ...)
   with City::CACHE_KEY = 'cities'.
2. PricingService::distanceKm(): cache the km lookup per pair with key
   "distance:{from}:{to}" for one week (road distances do not change).
3. Invalidation: in City::booted(), forget City::CACHE_KEY when a City
   is saved or deleted.
Tests: the second GET /api/cities runs zero queries (Cache uses the
array store in tests); updating a city's name_en changes the next
/api/cities response.
```

ما الذي تتحقق منه.

- جيد: يحتوي `.env` على `CACHE_STORE=redis`؛ ويُبقي `phpunit.xml` على `CACHE_STORE=array` حتى لا تلمس الاختبارات Redis.
- علامة تحذير: تخزين `GET /api/shipments` مؤقتاً، أو أي شيء خاص بكل مستخدم، من دون معرّف المستخدم في المفتاح. سيرى المستخدم A شحنات المستخدم B.
- علامة تحذير: لا إبطال للمدن، «لأن المدن لا تتغير أبداً». يجب أن يوجد الاختبار الذي يعيد تسمية مدينة. (المسافات هي الاستثناء المقصود الوحيد: إن غيّرتها يوماً، فشغّل `sail artisan cache:clear`.)

افحص Redis يدوياً:

```
sail redis redis-cli --scan --pattern '*cities*'
```

```
shahna-database-shahna-cache-cities
```

(تعتمد البادئة على `APP_NAME` و`CACHE_PREFIX` لديك.)

الخطوة 9 — حدود المعدّل ومعرّفات الطلبات (15 دقيقة)

ماذا ولماذا. يحميك تحديد المعدّل (Rate Limiting) من عميل معيب يدور في حلقة، ولاحقاً من شخص يضخّم فاتورة الذكاء الاصطناعي لديك. ومعرّفات الطلبات (Request IDs) تتيح لك تتبّع طلب واحد عبر السجلات والمهام في الطوابير.

🤖 **اسأل وكيلك الذكي**

الطلب للوكيل

خطّط أولاً، ثم انتظر موافقتي. الاختبارات أولاً.

1. في AppServiceProvider::boot عرِّف: RateLimiter::for('api', …) بحدّ 60 في الدقيقة حسب معرّف المستخدم، مع الرجوع إلى عنوان IP عند غيابه. طبّقه على جميع مسارات API عبر middleware->throttleApi() في bootstrap/app.php. (يأتي محدِّد "agent" أكثر صرامة في اليوم الخامس.)
2. برمجية وسيطة App، تُضاف في بداية المكدّس العام في bootstrap/app.php: اقرأ X-Request-Id أو ولّد UUID، وأضفه إلى كل سطر في السجل باستخدام Log::shareContext(['request_id' => …])، وأعِده في ترويسة الاستجابة X-Request-Id.

الاختبارات: الطلب رقم 61 إلى GET /api/shipments خلال دقيقة يعيد 429؛ والاستجابات تتضمن X-Request-Id؛ وقيمة X-Request-Id الواردة تُعاد كما هي.

بالإنجليزية:

```
Plan first, then wait for my approval. Tests first.

1. In AppServiceProvider::boot define:
   RateLimiter::for('api', 60 per minute by user id, falling back to IP)
   Apply it to all API routes via $middleware->throttleApi() in
   bootstrap/app.php. (A tighter "agent" limiter comes on Day 5.)
2. Middleware App\Http\Middleware\AssignRequestId, prepended to the
   global stack in bootstrap/app.php: read X-Request-Id or generate a
   UUID, add it to every log line with
   Log::shareContext(['request_id' => ...]), and return it as the
   X-Request-Id response header.
Tests: the 61st request to GET /api/shipments in a minute returns 429;
responses include X-Request-Id; an incoming X-Request-Id is echoed.
```

ما الذي تتحقق منه.

- جيد: `Limit::perMinute(60)->by($request->user()?->id ?: $request->ip())`.

- جيد: يستدعي اختبار 429 الدالة `RateLimiter::clear(...)` أو يستخدم مستخدمين جدداً حتى لا يتسرّب أثره إلى اختبارات أخرى.
- علامة تحذير: المحدِّد مفتاحه عنوان IP فقط. كل من في قاعة التدريب يتشاركون عنوان IP واحداً خلف الموجّه.

```
for i in $(seq 1 62); do
  curl -s -o /dev/null -w "%{http_code} " http://localhost/api/shipments \
    -H "Accept: application/json" -H "Authorization: Bearer $TOKEN"
done; echo
```

```
200 200 200 ... 200 429 429
```

انتظر دقيقة حتى يُعاد ضبط المحدِّد، ثم:

```
curl -s -i http://localhost/api/shipments -H "Accept: application/json" \
  -H "Authorization: Bearer $TOKEN" | grep -iE "x-ratelimit|x-request-id"
```

```
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 59
X-Request-Id: 9b2f0c1e-6a7d-4f0e-9b9e-2a1c3d4e5f60
```

الخطوة 10 — قياس k6 بعد التغيير، يدوياً (15 دقيقة)

> **تحذير**
>
> **يدوياً.** شغّل k6 واملأ الجدول بنفسك. ثم اطلب من الوكيل تفسير الأرقام، وقارن تفسيره بما غيّرته فعلاً.

انتظر دقيقة واحدة حتى يُعاد ضبط محدِّد المعدّل، ثم:

```
sail artisan migrate:fresh --seed
k6 run load/booking.js
```

أضف قسماً بعنوان "After" إلى `docs/plans/day3-perf.md`:

المقياس	خط الأساس	بعد	التغيّر
p95 لـ `list`			
p95 لـ `book`			
الاستعلامات لكل `GET /api/shipments` (الخطوة 2)	32		

المقياس	خط الأساس	بعد	التغيّر
`http_req_failed`			
الطلبات في الثانية			

النتيجة المعتادة: تنخفض قيمة p95 لـ `list` لأن الصفحة تنفّذ الآن عدداً ثابتاً من الاستعلامات؛ ويصبح `book` أسرع لأن الإسناد والبريد الإلكتروني انتقلا إلى الطوابير؛ ويبقى `http_req_failed` عند 0% لأن كل مستخدم يبقى تحت 60 طلباً في الدقيقة.

🤖 اسأل وكيلك الذكي

الطلب للوكيل

اقرأ docs/plans/day3-perf.md و git log day2-end..HEAD -oneline. لكل صف في الجدول، اشرح أي إيداع تسبّب على الأرجح في التغيير. إن لم يتحسن رقم أو ساء، فقل ذلك واقترح تجربة واحدة لمعرفة السبب. لا تغيّر الشيفرة. ضع علامة «فرضية» على كل ادعاء لا تستطيع إثباته من الأرقام.

بالإنجليزية:

```
Read docs/plans/day3-perf.md and `git log day2-end..HEAD --oneline`.
For each row of the table, explain which commit most likely caused the
change. If a number did not improve or got worse, say so and propose one
experiment to find out why. Do not change code. Mark every claim you
cannot prove from the numbers as a hypothesis.
```

إن ادّعى الوكيل تحسناً لا يُظهره جدولك، فهو مخطئ. دوّن ذلك من أجل التأمل الختامي.

نقطة تحقق

نقطة التحقق 2 (15:45)

```
sail artisan test
```

```
  Tests:    75+ passed
```

يحتوي المرجع `day3-end` على 80 اختباراً؛ وسيكون عددك قريباً من ذلك.

```
# Missing idempotency key
curl -s -X POST http://localhost/api/shipments -H "Accept: application/json" \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" -d '{}'
```

```
{"message":"The Idempotency-Key header is required."}
```

```
# Customer cannot use the dispatcher route
curl -s -X POST http://localhost/api/dispatch/shipments/$REF/assign \
  -H "Accept: application/json" -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" -d '{"vehicle_id":1}'
```

```
{"message":"This action is unauthorized."}
```

لوحة Horizon: صفر مهام فاشلة. ويحتوي `docs/plans/day3-perf.md` على الجدولين مملوءين.

```
git add -A
git commit -m "feat: idempotency, caching, rate limits, perf report"
```

إذا تعثّرت

الحلّ المرجعي هو الوسم (Tag) `day3-end`.

```
git diff --stat day3-end
git show day3-end:app/Jobs/AssignVehicleJob.php
git diff day3-end -- bootstrap/app.php
git show day3-end:load/booking.js > /tmp/booking-ref.js && k6 run /tmp/booking-ref.js
```

مشكلات شائعة:

العَرَض	السبب المرجّح
المهام تبقى في "Pending" في Horizon	Horizon لا يعمل، أو `QUEUE_CONNECTION` لا تزال `sync` / `database`
المهمة تفشل مع "No query results for model"	أُرسل الحدث قبل إيداع المعاملة
تغييرك في الشيفرة لا أثر له على المهام	لم يُعَد تشغيل Horizon
يُظهر k6 أخطاء 429 كثيرة	`sleep(3)` مفقود، أو كل المستخدمين الافتراضيين يتشاركون رمزاً واحداً
كل إعادة محاولة لحجز تحصل على 409	تعطّل طلب بعد حجز المفتاح؛ يُستعاد المفتاح بعد 60 ثانية
k6 في Docker: الاتصال مرفوض (connection refused)	استخدم `BASE_URL=http://host.docker.internal`

لإعادة البدء من الحلّ المرجعي:

```
git stash
git switch -c my-day3-reset day3-end
sail composer install
sail artisan migrate:fresh --seed
sail artisan test
```

أهداف إضافية

1. **SKIP LOCKED.** غيّر استعلام المركبة إلى `FOR UPDATE SKIP LOCKED` (`->lock('for update skip')`) `locked')` حتى يختار العامل الثاني المركبة الشاغرة *التالية* بدلاً من الانتظار. اشرح متى يكون ذلك أفضل ومتى يكون أسوأ.
2. **Pulse.** ثبّت Laravel Pulse وجِد أبطأ استعلام وأبطأ مهمة لديك في لوحته أثناء تشغيل k6.
3. **اختبار الذروة (Spike Test).** أضف سيناريو k6 ثانياً يستخدم المنفّذ `ramping-vus` ويرتفع من 0 إلى 100 مستخدم افتراضي خلال 30 ثانية. أين ينهار النظام أولاً: PHP، أم اتصالات PostgreSQL، أم Redis؟
4. **معالجة المهام الفاشلة.** أضف دالة `failed()` إلى `AssignVehicleJob` تسجّل ملاحظة `ShipmentEvent` بنص "Automatic assignment failed" وتكتب مرجع الشحنة في السجل.
5. **معرّفات الطلبات في المهام.** يغطي `Log::shareContext()` طلب HTTP فقط. حوّل `AssignRequestId` إلى الواجهة `Context` في Laravel حتى يظهر معرّف الطلب أيضاً في سجلات المهام التي وضعها ذلك الطلب في الطابور.

التأمل

1. ما ادعاء الأداء الذي قدّمه الوكيل اليوم ولم تدعمه أرقام k6 لديك؟ إن لم يوجد، فأي ادعاء تحققت منه، وكيف؟
2. اشرح بكلماتك لماذا يجب إدراج صف عدم التكرار *قبل* تشغيل المتحكم. ما الاختبار الذي سيكشف تنفيذاً من نوع «تحقّق ثم أدرِج»، وهل كتبته؟
3. قفلتَ صف المركبة في `AssignVehicleJob`. صِف بالضبط الإسناد المزدوج الذي يحدث من دون القفل، باستخدام عاملين وشاحنة صغيرة واحدة في بنغازي.